STAT340 - Topic 6: Logistic Regression

Logistic regressions

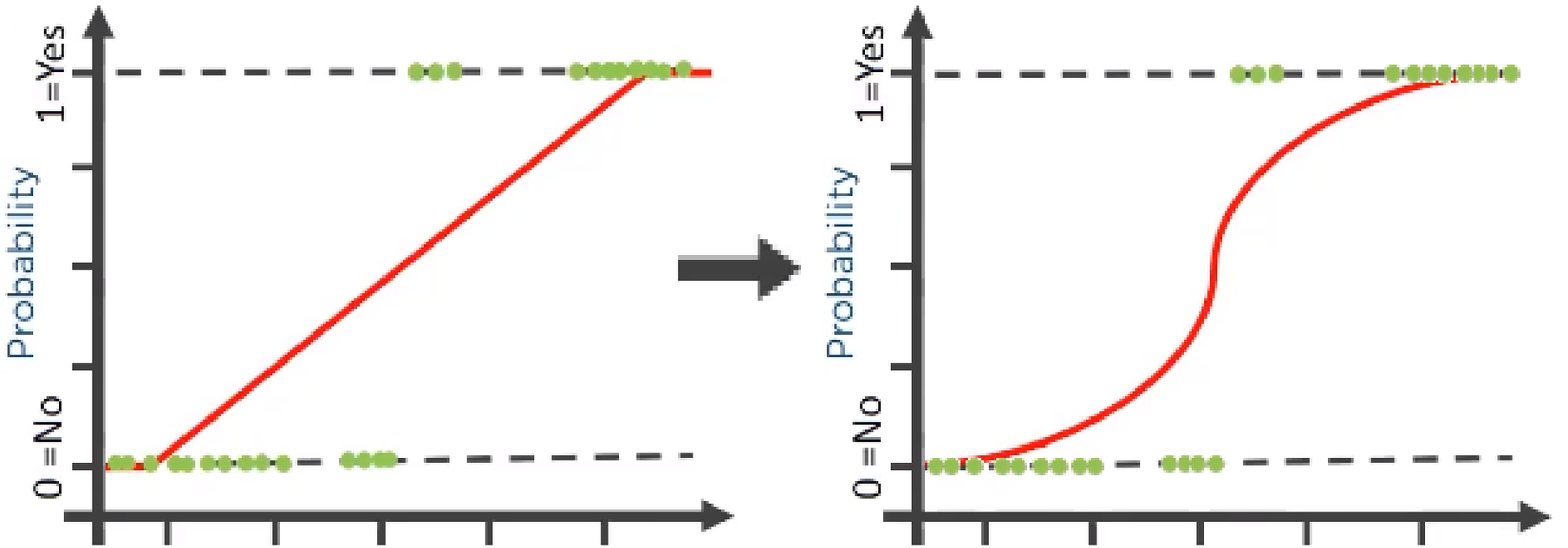
What if response variable is binary categorical? E.g. yes/no, success/failure, A/B.

- Can represent responses as 1/0 and consider regressing **probability of 1 (success)**, but need prediction to be constrained to [0,1]
- Direct SLR/MLR not suitable since predicted response can NOT be limited to [0,1]
- Solution: fit **transformation of predicted probability of 1** with output range in [0,1]
- Most commonly, we fit **logistic function**—i.e. **log-odds**—of estimated probability of 1

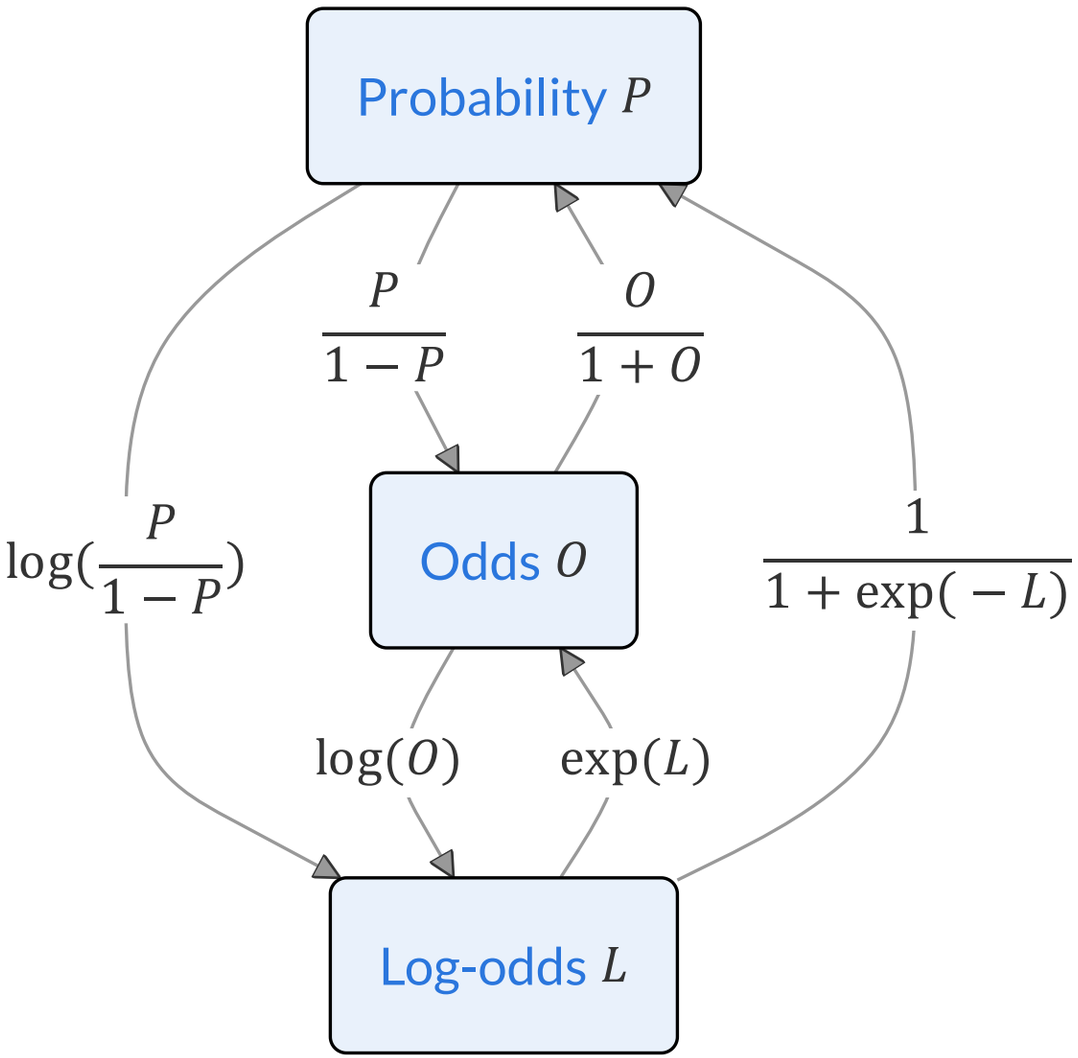
Logistic regressions

What if response variable is binary categorical? E.g. yes/no, success/failure, A/B.

- Can represent responses as 1/0 and consider regressing **probability of 1 (success)**, but need prediction to be constrained to [0,1]
- Direct SLR/MLR not suitable since predicted response can NOT be limited to [0,1]
- Solution: fit **transformation of predicted probability of 1** with output range in [0,1]
- Most commonly, we fit **logistic function**—i.e. **log-odds**—of estimated probability of 1

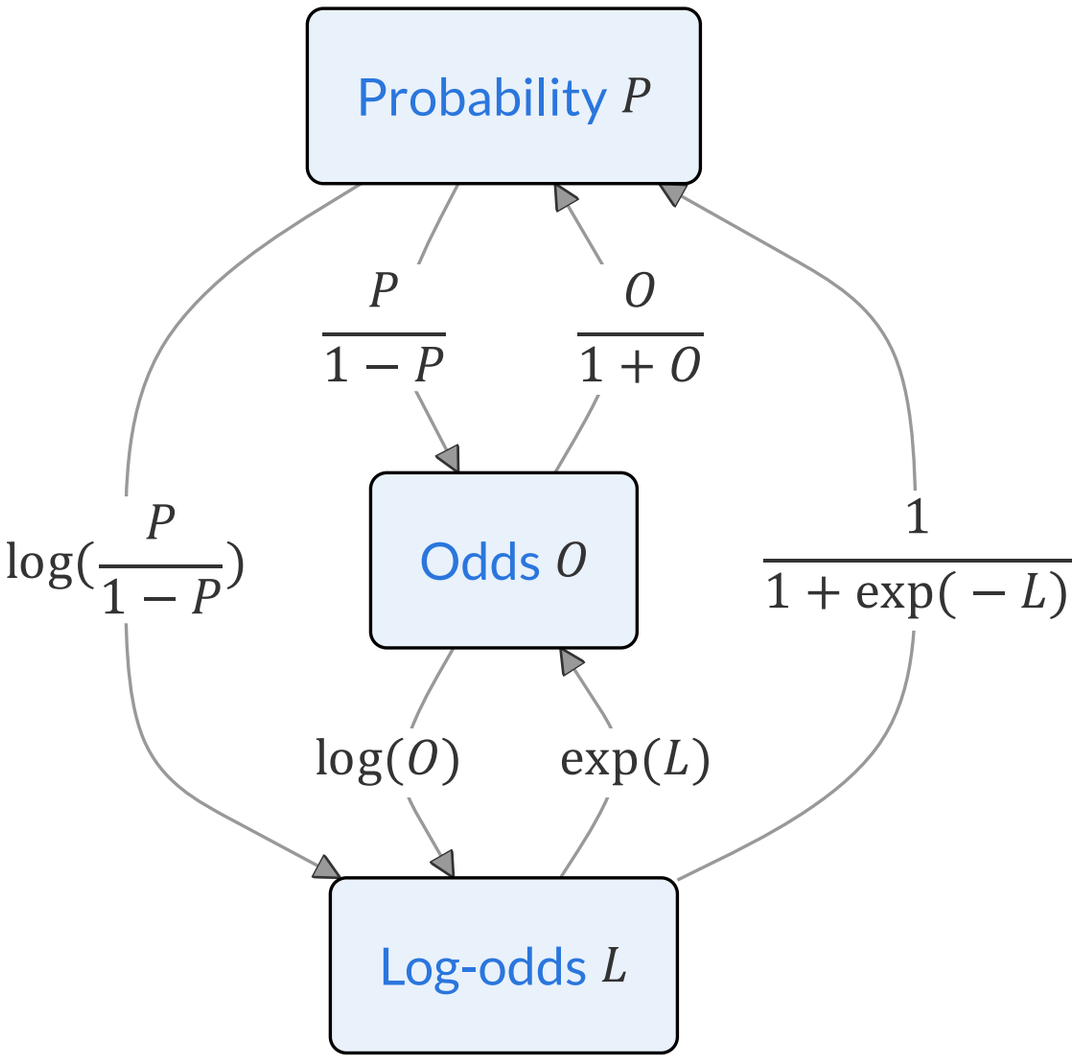


Probability, odds, log odds



- Probability P: $[\ 0 \longleftarrow 0.5 \longrightarrow 1\ ]$
- Odds O: $[\ 0 \longleftarrow 1 \longrightarrow \infty\ )$
- Log-odds L: $(-\infty \longleftarrow 0 \longrightarrow \infty\ )$
- Odds are **multiplicative**, but log-odds are **additive**, increase in log-odds means increase in probability & vice versa

Probability, odds, log odds



- Probability P: $[\ 0 \longleftarrow 0.5 \longrightarrow 1\ ]$
- Odds O: $[\ 0 \longleftarrow 1 \longrightarrow \infty\ )$
- Log-odds L: $(-\infty \longleftarrow 0 \longrightarrow \infty\ )$
- Odds are **multiplicative**, but log-odds are **additive**, increase in log-odds means increase in probability & vice versa

Also note, **logistic function** refers to:

$$f(x) = \frac{1}{1+\exp(-x)}$$

and its inverse, called **logit function**:

$$f^{-1}(x) = \log\left(\frac{x}{1-x}\right)$$

Note: don't forget sign in $\exp(-x)$ term

Logistic regression model

For each predictor value x, suppose the true probability of 1 there, $p(x)$ is:

$$p(x) = \frac{1}{1 + \exp[-(\beta_0 + \beta_1 x)]}$$

Note this is equivalent to:

$$\log\left(\frac{p(x)}{1 - p(x)}\right) = \beta_0 + \beta_1 x$$

Logistic regression model

For each predictor value x, suppose the true probability of 1 there, $p(x)$ is:

$$p(x) = \frac{1}{1 + \exp[-(\beta_0 + \beta_1 x)]}$$

Note this is equivalent to:

$$\log\left(\frac{p(x)}{1 - p(x)}\right) = \beta_0 + \beta_1 x$$

- Best fit $\hat{\beta}_0$, $\hat{\beta}_1$ are found numerically (no closed form solution), demo
  - Can be easily extended to MLR by adding $+ \beta_2 x_{i,2} + \cdots + \beta_p x_{i,p}$
  - Can of course add categoricals, interactions, nonlinear terms, etc..
- Thus, logistic regression equivalent to **SLR/MLR** with **log-odds of 1 as response**.
- This means **coefficients** in summary table correspond to **log odds**, i.e:
  - Intercept is estimated baseline log-odds of 1 at $x = 0$,
  - Slope is estimated increase in log-odds for each 1 unit increase in x

Important differences

Besides transforming to log-odds, there are some other differences:

- Logistic regression usually fitted by MLE (maximum likelihood estimation), not OLS
- Logistic regression is a type of GLM (generalized linear model), where transformation of mean is fitted to predictors

Important differences

Besides transforming to log-odds, there are some other differences:

- Logistic regression usually fitted by MLE (maximum likelihood estimation), not OLS
- Logistic regression is a type of GLM (generalized linear model), where transformation of mean is fitted to predictors

- For logistic regression, equation directly models mean log-odds with predictors, not each individual 1/0 y_i response, so **no error ε_i terms**
  - Thus, **residuals NOT assumed to be N(0,σ^2)**, and are not important to us
- When predicting, use coefficients similarly to MLR to get **predicted log-odds of 1**, then can convert to probability if desired
- In summary, all statistics are Z-based (i.e. normal) instead of T-based
  - I.e. instead of T-tests and T-intervals, do the equivalent Z-tests and Z-intervals

Example: Credit `Default` dataset

The `ISLR2` dataset `Default` contains (simulated) credit default data of customers:

```
# shuffle rows to better show categorical levels, then save and preview
Default = slice_sample(ISLR2::Default, prop=1)  ;  str(Default)
```

```
'data.frame':   10000 obs. of  4 variables:
 $ default: Factor w/ 2 levels "No","Yes": 1 1 2 1 1 2 1 1 1 1 ...
 $ student: Factor w/ 2 levels "No","Yes": 1 2 1 1 1 1 1 1 1 1 ...
 $ balance: num  939 398 1512 301 878 ...
 $ income : num  45519 22711 53507 51540 29562 ...
```

```
head(Default, 3)
```

```
  default student   balance   income
1      No      No  939.0985 45519.02
2      No     Yes  397.5425 22710.87
3     Yes      No 1511.6110 53506.94
```

Example: Credit `Default` dataset

The `ISLR2` dataset `Default` contains (simulated) credit default data of customers:

```
# shuffle rows to better show categorical levels, then save and preview
Default = slice_sample(ISLR2::Default, prop=1)  ;  str(Default)
```

```
'data.frame':   10000 obs. of  4 variables:
 $ default: Factor w/ 2 levels "No","Yes": 1 1 2 1 1 2 1 1 1 1 ...
 $ student: Factor w/ 2 levels "No","Yes": 1 2 1 1 1 1 1 1 1 1 ...
 $ balance: num  939 398 1512 301 878 ...
 $ income : num  45519 22711 53507 51540 29562 ...
```

```
head(Default, 3)
```

```
  default student   balance   income
1      No      No  939.0985 45519.02
2      No     Yes  397.5425 22710.87
3     Yes      No 1511.6110 53506.94
```

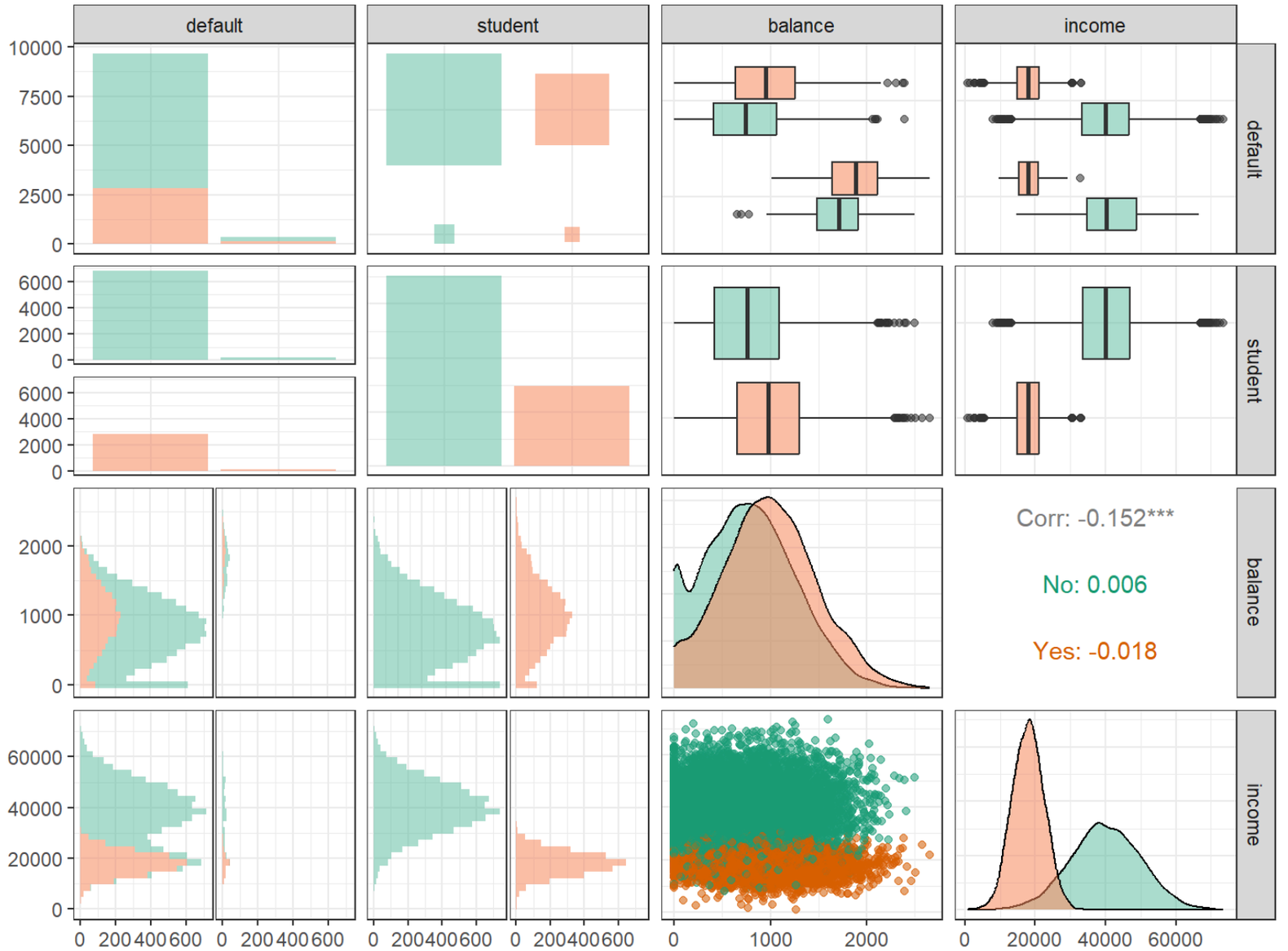
Let's predict which customers default, i.e. do not pay loans back, letting 1 be defaults

```
# convert default factor to 1/0
Default$default = ifelse(Default$default == "Yes", 1, 0)
head(Default, 3)
```

```
  default student   balance   income
1       0      No  939.0985 45519.02
2       0     Yes  397.5425 22710.87
3       1      No 1511.6110 53506.94
```

Example: Credit `Default` dataset (cont.)

```
GGally::ggpairs(ISLR2::Default,aes(color=student,alpha=.9)) # quick plot
```



Example: Credit `Default` dataset (cont.)

```r
# fit logistic model with glm(), remember to have family=binomial!!
glm.default = glm(default ~ . , data=Default, family=binomial)
# only print important rows, note deviance is a loss metric, but we won't worry about it
summary(glm.default)  %>% {capture.output(.)[5:12]} %>% cat(sep="\n")
```

```
Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.087e+01  4.923e-01 -22.080  < 2e-16 ***
studentYes  -6.468e-01  2.363e-01  -2.738  0.00619 **
balance      5.737e-03  2.319e-04  24.738  < 2e-16 ***
income       3.033e-06  8.203e-06   0.370  0.71152
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Example: Credit `Default` dataset (cont.)

```r
# fit logistic model with glm(), remember to have family=binomial!!
glm.default = glm(default ~ . , data=Default, family=binomial)
# only print important rows, note deviance is a loss metric, but we won't worry about it
summary(glm.default)  %>% {capture.output(.)[5:12]} %>% cat(sep="\n")
```

```
Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.087e+01  4.923e-01 -22.080  < 2e-16 ***
studentYes  -6.468e-01  2.363e-01  -2.738  0.00619 **
balance      5.737e-03  2.319e-04  24.738  < 2e-16 ***
income       3.033e-06  8.203e-06   0.370  0.71152
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Interpretations:

- All else equal, **non-students & high-balances** both significantly **higher risk** of default
  - Non-students with 0 balance & income very low risk of default (log-odds $\approx -1.1\times10^{1}$)
  - All else equal, being **student decreases** log-odds of default by $\mathbf{-6.5\times10^{-1}}$
  - All else equal, every $1 remaining **balance increases** log-odds of default by $\mathbf{5.7\times10^{-3}}$
  - Income not significant at all, accounting for other variables

Exercise

Suppose we have a student with $2000 balance and $15,000 income. What are the predicted **log-odds**, **odds**, and **probability** of defaulting? Compute both manually & in R.

Exercise

Suppose we have a student with $2000 balance and $15,000 income. What are the predicted **log-odds**, **odds**, and **probability** of defaulting? Compute both manually & in R.

Answer:

```
# log-odds of defaulting = intercept + student coef + (2000 * balance coef) + (15000 * income coef)
(L = -1.087e1 + -6.468e-1 + 2000*5.737e-3 + 15000*3.033e-6)
```

```
[1] 0.002695
```

```
(O = exp(L))          # odds of defaulting
```

```
[1] 1.002699
```

```
(P = O/(1+O))         # probability of defaulting
```

```
[1] 0.5006737
```

```
(P = 1/(1+exp(-L)))  # alternative formula for p
```

```
[1] 0.5006737
```

```
# use R to predict, note by default R gives log-odds!  see help page of predict.glm() for details
predict(glm.default, newdata = tibble(student="Yes", balance=2e3, income=1.5e4))
```

```
[1] 0.002691261
```

```
# tell R to output prediction as probability instead
predict(glm.default, newdata = tibble(student="Yes", balance=2e3, income=1.5e4), type="response")
```

```
[1] 0.5006728
```

Testing table revisited:

Next, can take predicted probabilities above a threshold (often 0.5) as predicted 1, otherwise 0. These can be compared to true 1/0 values to get **confusion matrix**:

```
# easy R method: first get vector of true 1/0 values and convert to factor, a type of categorical rep
true.responses = Default$default %>% as.factor
# then, threshold predictions and convert first to numeric 1/0 values then to factor as well
pred.responses = (predict(glm.default,type="response") > 0.5) %>% as.numeric %>% as.factor

# use confusionMatrix() from caret package, set predicted/true values and what's "positive"
caret::confusionMatrix(data=pred.responses, reference=true.responses, positive="1") %>%
  {capture.output(.)[c(1:8,17:21)]} %>% cat(sep="\n")
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 9627  228
         1   40  105

               Accuracy : 0.9732
            Sensitivity : 0.3153
            Specificity : 0.9959
         Pos Pred Value : 0.7241
         Neg Pred Value : 0.9769
             Prevalence : 0.0333
```

Testing table revisited:

- Prediction 1 is positive, prediction 0 is negative
- If prediction = reference, then true, else false
- Thus TN=9627, FN=228, FP=40, TP=105

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 9627  228
         1   40  105

               Accuracy : 0.9732
                  Sensitivity : 0.3153
                  Specificity : 0.9959
               Pos Pred Value : 0.7241
               Neg Pred Value : 0.9769
                   Prevalence : 0.0333
```

Testing table revisited:

- Prediction 1 is positive, prediction 0 is negative
- If prediction = reference, then true, else false
- Thus TN=9627, FN=228, FP=40, TP=105

In table below confusion matrix,

- Accuracy (% pred. correct) = (TP+TN)/(TP+TN+FN+FP)
- Sensitivity (also power or TP rate) = TP/(TP+FN)
- Specificity (also TN rate or $1 - \alpha$) = TN/(TN+FP)
  - Note: this means $\alpha = 1$—TN/(TN+FP) = FP/(TN+FP)

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 9627  228
         1   40  105

               Accuracy : 0.9732
                 Sensitivity : 0.3153
                 Specificity : 0.9959
              Pos Pred Value : 0.7241
              Neg Pred Value : 0.9769
                  Prevalence : 0.0333
```

Testing table revisited:

- Prediction 1 is positive, prediction 0 is negative
- If prediction = reference, then true, else false
- Thus TN=9627, FN=228, FP=40, TP=105

In table below confusion matrix,

- Accuracy (% pred. correct) = (TP+TN)/(TP+TN+FN+FP)
- Sensitivity (also power or TP rate) = TP/(TP+FN)
- Specificity (also TN rate or $1-\alpha$) = TN/(TN+FP)
  - Note: this means $\alpha = 1$—TN/(TN+FP) = FP/(TN+FP)
- Positive predictive value (also precision) = TP/(TP+FP)
  - This is probability that a prediction of 1 is correct
- Negative predictive value = TN/(TN+FN)
  - This is probability that a prediction of 0 is correct
- Prevalence (% of 1s in population) = (TP+FN)/(TP+FN+TN+FP)

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 9627  228
         1   40  105

               Accuracy : 0.9732
                 Sensitivity : 0.3153
                 Specificity : 0.9959
              Pos Pred Value : 0.7241
              Neg Pred Value : 0.9769
                  Prevalence : 0.0333
```

Exercise

Consider confusion matrix below for predicting diabetes in the Pima Indians dataset. Compute accuracy, sensitivity, specificity, PPV, NPV, and prevalence.

	No diabetes	Yes diabetes
Predict 0	200	53
Predict 1	23	56

Exercise

Consider confusion matrix below for predicting diabetes in the Pima Indians dataset. Compute accuracy, sensitivity, specificity, PPV, NPV, and prevalence.

	No diabetes	Yes diabetes
Predict 0	200	53
Predict 1	23	56

```
TN = 200 ; FN = 53 ; FP = 23 ; TP = 56
(accuracy = (TP+TN)/(TP+TN+FP+FN))
```

```
[1] 0.7710843
```

```
(sensitivity = TP/(TP+FN))
```

```
[1] 0.5137615
```

```
(specificity = TN/(TN+FP))
```

```
[1] 0.896861
```

```
(PPV = TP/(TP+FP))
```

```
[1] 0.7088608
```

```
(NPV = TN/(TN+FN))
```

```
[1] 0.7905138
```

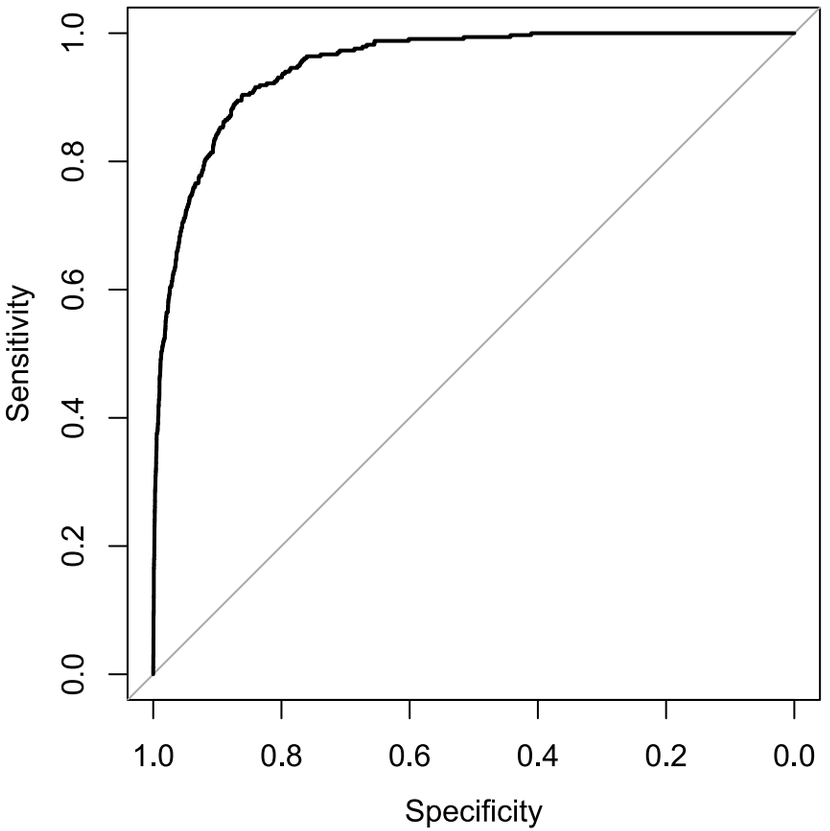
```
(prevalence = (TP+FN)/(TP+FN+TN+FP))
```

```
[1] 0.3283133
```

ROC curve

Receiver Operating Characteristic (ROC) curve visualizes model stats **across all thresholds** not just 0.5. For each threshold, **sensitivity vs specificity** plotted.

```
# easiest way in R: use roc(), plot.roc() functions from pROC package
library(pROC)
pred.probabilities = predict(glm.default,type="response")
roc(true.responses, pred.probabilities) %>% plot.roc
```



ROC curve

Receiver Operating Characteristic (ROC) curve visualizes model stats **across all thresholds** not just 0.5. For each threshold, **sensitivity vs specificity** plotted.

```
# easiest way in R: use roc(), plot.roc() functions from pROC package
library(pROC)
pred.probabilities = predict(glm.default,type="response")
roc(true.responses, pred.probabilities) %>% plot.roc
```

- More ROC curve "arches" into top left corner, the better the model is
  - Null model represented by 45° angle line
- Entire Area Under Curve (AUC) equals chance model can distinguish a 1 vs a 0
  - AUC = 0.5 for null model
  - AUC = 1 for perfect model

```
auc(true.responses, pred.probabilities)
```

```
Area under the curve: 0.9496
```

- Nice shiny demo with Pima Indians dataset

